FULL-STACK ENGINEERING & SYSTEM ARCHITECTURE BIBLE

StayLust

Complete Project Notes, Code Implementations & Viva Guide

An exhaustive engineering dossier detailing the design, implementation, and architectural trade-offs of **StayLust** — a production-grade accommodation booking and vacation rental web application powered by React 19, Express 5, MongoDB Atlas, Mapbox GL JS, and Cloudinary.

AUTHOR & DEVELOPER

Ranjit Sharma

ARCHITECTURE MODEL

Decoupled SPA + RESTful API (MVC)

FULL TECH STACK

React 19 • Vite 8 • Tailwind v4 • Node 24 • Express 5 • Atlas

CLOUD SERVICES

Cloudinary CDN • Mapbox GL Geocoding • Render CI/CD

GITHUB REPOSITORY

github.com/ranjit-sharma/StayLust

LIVE DEPLOYMENT

wonderlust-yppn.onrender.com

Framework: What • Why • Implementation (Code & Visuals)

Confidential & Comprehensive Major Project Documentation

Chapter 1: System Architecture & Decoupled SPA Paradigm

📌 WHAT IT IS

A **Decoupled Single Page Application (SPA) + REST API architecture**. The client-side interface is an autonomous React 19 application compiled by Vite, while the server is an Express.js 5 micro-monolith connected to MongoDB Atlas. Express acts strictly as an API gateway returning JSON and serving precompiled static production bundles.

💡 WHY WE BUILT IT THIS WAY

Traditional monolithic Multi-Page Applications (MPAs) using EJS or Handlebars trigger a full document reload on every action. This introduces white flashes, destroys client-side scroll position, causes WebGL map canvases (Mapbox) to re-initialize from scratch on every page turn, and places high rendering CPU strain on Node.js. By decoupling into an SPA:

- Client state is preserved across route changes (seamless navigation).
- Network payloads are reduced to lean JSON strings rather than redundant HTML structures.
- Optimistic UI reconciliation becomes possible (0ms perceived latency for deletions).

⚙️ IMPLEMENTATION & CODE WALKTHROUGH

1. Development Proxy (client/vite.config.js):

CLIENT/VITE.CONFIG.JS

```
import { defineConfig } from "vite";
import react from "@vitejs/plugin-react";
import tailwindcss from "@tailwindcss/vite";

export default defineConfig({
  plugins: [react(), tailwindcss()],
  server: {
    // Express runs on 8080; proxying keeps the session cookie same-origin in development
    proxy: { "/api": "http://localhost:8080" },
  },
});
```

2. Production Serving & Catch-All Routing (app.js):

APP.JS

```
const clientDist = path.join(__dirname, "client", "dist");

app.use(express.json());
// Serve compiled assets before session middleware to prevent asset session creation
app.use(express.static(clientDist));

// REST API Routers
app.use("/api/listings", listingRouter);
app.use("/api/listings/:id/reviews", reviewRouter);
app.use("/api/listings/:id/bookings", bookingRouter);
app.use("/api", userRouter);
app.use("/api", infoRouter);

// Catch-all route delegating client routing to React Router
app.get("/*splat", (req, res) => {
  res.sendFile(path.join(clientDist, "index.html"));
});
```

Chapter 2: Authentication & Distributed Session Management

📌 WHAT IT IS

Stateful session-based user authentication powered by **Passport.js Local Strategy**, salted **PBKDF2 password hashing**, and a distributed session store backed by **connect-mongo** inside MongoDB Atlas.

💡 WHY WE BUILT IT THIS WAY

Many student projects use JSON Web Tokens (JWT) stored in `localStorage`. This introduces major security vulnerabilities:

- **XSS Vulnerability:** Any malicious third-party script or npm package can access `localStorage` and steal the token.
- **Inability to Revoke:** JWTs cannot be invalidated on demand unless you build a complex Redis blacklist. If a host terminates a session or account, a stolen JWT remains valid until expiration.
- **Session Cookies:** StayLust uses `httpOnly: true` and `secure: true` cookies. JavaScript cannot access them, eliminating XSS token theft. Sessions can be destroyed instantly on the server via `req.session.destroy()`.

⚙️ IMPLEMENTATION & CODE WALKTHROUGH

APP.JS: DISTRIBUTED MONGODB SESSION STORE

```
const store = MongoStore.create({
  mongoUrl: dbUrl,
  crypto: { secret: process.env.SESSION_SECRET },
  touchAfter: 24 * 3600, // Lazy session update: only rewrite if session changed within 24h
});

const sessionOptions = {
  store,
  secret: process.env.SESSION_SECRET,
  resave: false,
  saveUninitialized: false,
  cookie: {
    expires: Date.now() + 7 * 24 * 60 * 60 * 1000,
    maxAge: 7 * 24 * 60 * 60 * 1000,
    httpOnly: true, // Prevents client-side JS from accessing the cookie
    secure: process.env.NODE_ENV === "production",
    sameSite: "lax",
  },
};

app.use(session(sessionOptions));
app.use(passport.initialize());
app.use(passport.session());
passport.use(new LocalStrategy(User.authenticate()));
passport.serializeUser(User.serializeUser());
passport.deserializeUser(User.deserializeUser());
```

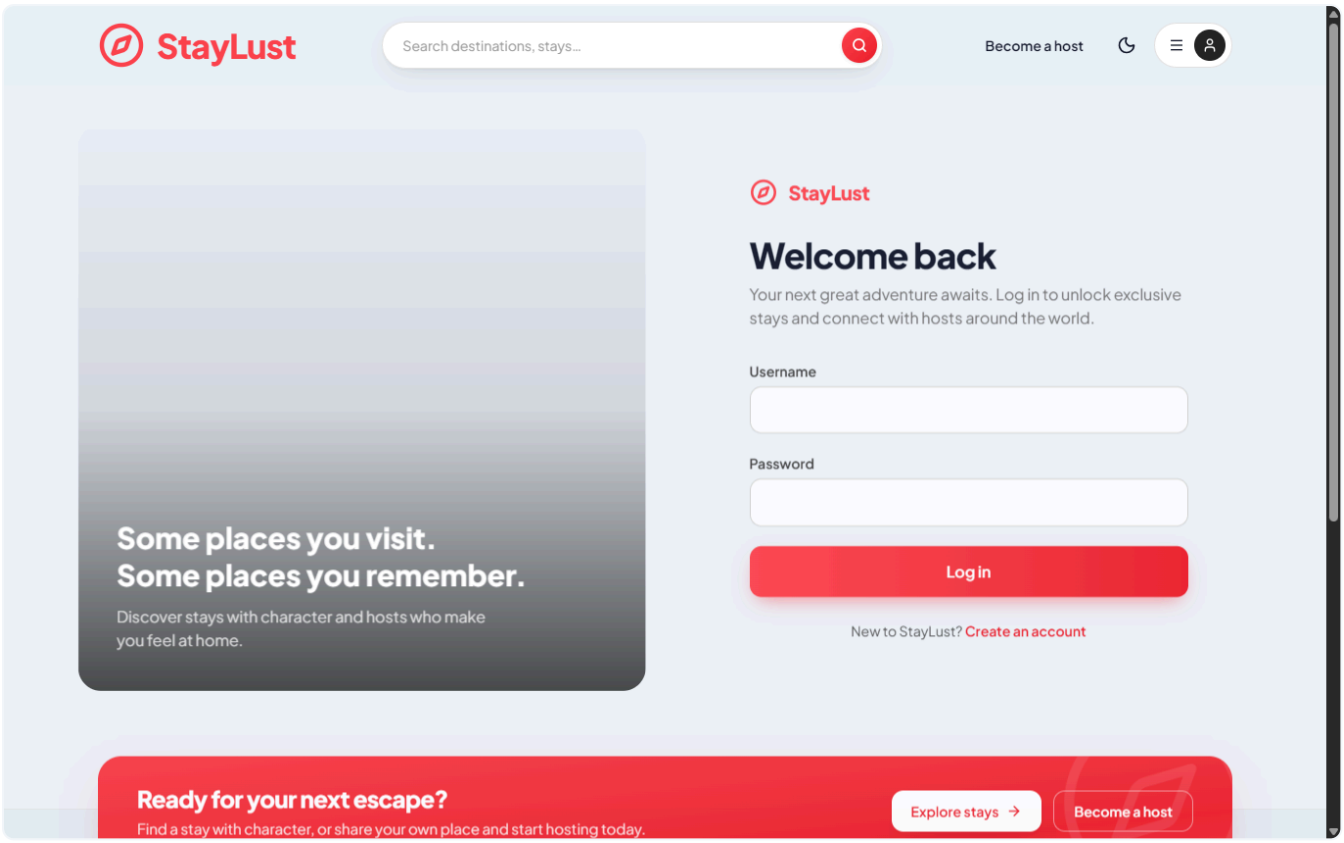


Figure 2.1: Secure Split-Screen Authentication Portal with PBKDF2 Encryption.

Chapter 3: Dynamic 4-Row Grid & Card Hierarchy

📌 WHAT IT IS

A viewport-adaptive accommodation grid that dynamically computes **exactly 4 complete rows of property cards** across all device types, with a single-click "Show more stays" accordion and dedicated photo-only navigation.

💡 WHY WE BUILT IT THIS WAY

Hardcoded pagination (e.g., showing 12 stays) looks asymmetrical: on a 6-column widescreen desktop it yields only 2 rows, while on a 2-column mobile device it generates an overwhelming 6 rows. By calculating `cols * 4` dynamically:

- **Mobile (<768px):** 2 cols × 4 rows = **8 stays**.
- **Tablet Portrait (768px-1023px):** 3 cols × 4 rows = **12 stays**.
- **Tablet Landscape (1024px-1279px):** 4 cols × 4 rows = **16 stays**.
- **Desktop Widescreen (≥1280px):** 6 cols × 4 rows = **24 stays**.
- **Navigation Isolation:** The `<Link>` tag wraps *strictly the photo*. Users can highlight address text or click wishlist buttons without triggering accidental page transitions.

⚙ IMPLEMENTATION & CODE WALKTHROUGH

CLIENT/SRC/PAGES/LISTINGS.JSX: RESPONSIVE CALCULATOR

```
// Dynamic responsive columns: 6 on desktop (xl), 4 on landscape tablet (lg), 3 on portrait tablet (md), 2 on
const isXl = useMediaQuery("(min-width: 1280px)");
const isLg = useMediaQuery("(min-width: 1024px)");
const isMd = useMediaQuery("(min-width: 768px)");
const cols = isXl ? 6 : isLg ? 4 : isMd ? 3 : 2;
const initialListingCount = cols * 4; // Exactly 4 complete lines of listings

const visibleListings = showAll ? allListings : allListings.slice(0, initialListingCount);
```

CLIENT/SRC/COMPONENTS/LISTINGCARD.JSX: ISOLATED PHOTO LINK & INLINE RATING

```
{/* Navigation happens strictly on clicking the photo */}
<Link
  to={`/listings/${listing._id}`}
  aria-label={listing.title}
  className="block relative aspect-[4/3.6] overflow-hidden rounded-card bg-neutral-200"
>
  <SmoothImage src={imageUrl(listing.image?.url, 600)} alt={listing.title} />
</Link>

<div className="mt-2 sm:mt-3 px-0.5 sm:px-1">
  {/* Line 1: Highlighted Title */}
  <h3 className="truncate font-bold text-xs sm:text-base text-neutral-900">{listing.title}</h3>

  {/* Line 2: Muted Address */}
  <p className="truncate text-[11px] sm:text-sm text-neutral-500">
    {[listing.location, listing.country].filter(Boolean).join(", ")}
  </p>

  {/* Line 3: Nightly Price + Inline Star Rating */}
  <p className="mt-0.5 flex items-center gap-1 sm:gap-1.5 text-xs sm:text-[15px] truncate">
    <span className="font-bold">{formatPrice(price)}</span>
    <span className="text-neutral-500 text-[11px] sm:text-sm">night</span>
    <span className="text-neutral-400 text-[11px] select-none">·</span>
    <Rating value={listing.averageRating} count={listing.reviewCount} showCount={false} />
  </p>
</div>
```

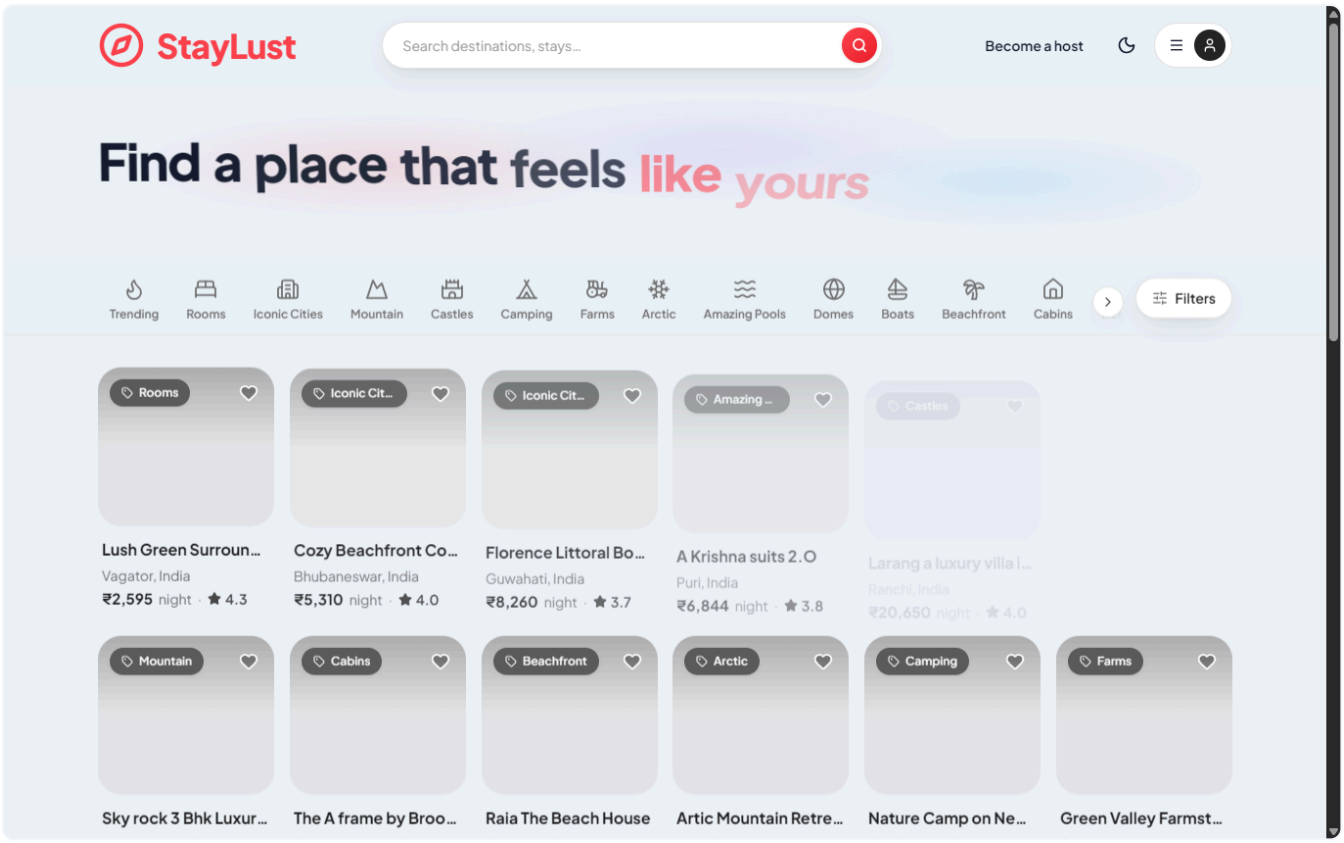


Figure 3.1: StayLust Homepage showing the 4-row grid, enlarged logo, and inline card rating.

Chapter 4: Default 18% GST Pricing Engine

📌 WHAT IT IS

A transparent pricing computation layer that automatically factors India's statutory **18% Goods and Services Tax (GST)** into all displayed listing prices, reservation calculations, and booking payout summaries by default.

💡 WHY WE BUILT IT THIS WAY

Mutating raw price fields directly inside MongoDB (e.g. saving ₹2,595 instead of ₹2,199) violates standard database normalization:

- If tax laws change (e.g. from 18% to 12%), an entire database migration would be required.
- Hosts would receive skewed commission reports because base payouts would be conflated with government tax liabilities.
- StayLust stores the raw base price in MongoDB and applies `const price = Math.round((Number(listing.price) || 0) * (1 + GST_RATE))` on the presentation layer, keeping base data pristine while giving guests accurate checkout figures.

⚙️ IMPLEMENTATION & CODE WALKTHROUGH

CLIENT/SRC/LIB/FORMAT.JS: CENTRALIZED FINANCIAL LOGIC

```
export const GST_RATE = 0.18; // India accommodation tax standard rate

export function formatPrice(amount) {
  if (amount == null || isNaN(amount)) return "₹0";
  return new Intl.NumberFormat("en-IN", {
    style: "currency",
    currency: "INR",
    maximumFractionDigits: 0,
  }).format(amount);
}

export function bookingTotals(nightlyPrice, nights) {
  const base = (Number(nightlyPrice) || 0) * (Number(nights) || 0);
  const gst = Math.round(base * GST_RATE);
  const total = base + gst;
  return { base, gst, total };
}
```

Chapter 5: Cloudinary Media Storage & Multer Upload Pipeline

📌 WHAT IT IS

A cloud-native media streaming pipeline that connects Express multipart form uploads directly to **Cloudinary CDN** via **Multer**, storing secure HTTPS URLs and public asset IDs in MongoDB.

💡 WHY WE BUILT IT THIS WAY

Writing file uploads to local disk (e.g. `/public/uploads`):

- Fails on cloud deployment platforms (Render, Heroku, AWS Lambda) because server file systems are *ephemeral*—any container restart wipes out all user-uploaded photos.
- Slows down response times because Node.js must serve multi-megabyte image assets instead of delegating to a dedicated Content Delivery Network.
- Cloudinary provides automatic format negotiation (serving modern WebP and AVIF to supported browsers) and automated image resizing.

⚙️ IMPLEMENTATION & CODE WALKTHROUGH

CLOUDCONFIG.JS: CLOUDINARY STORAGE ENGINE

```
const cloudinary = require('cloudinary').v2;
const { CloudinaryStorage } = require('multer-storage-cloudinary');

cloudinary.config({
  cloud_name: process.env.CLOUD_NAME,
  api_key: process.env.CLOUD_API_KEY,
  api_secret: process.env.CLOUD_API_SECRET,
});

const storage = new CloudinaryStorage({
  cloudinary: cloudinary,
  params: {
    folder: 'staylust_DEV',
    allowedFormats: ["png", "jpg", "jpeg", "webp", "avif"],
  },
});

module.exports = { cloudinary, storage };
```

ROUTES/LISTING.JS: ROUTE HANDLER WITH MULTER MIDDLEWARE

```
const upload = multer({ storage });

router.route("/")
  .get(wrapAsync(listingController.index))
  .post(
    isLoggedIn,
    upload.single("listing[image]"), // Streams binary file to Cloudinary
    validateListing,                 // Validates body with Joi schema
    wrapAsync(listingController.createListing)
  );
```

Chapter 6: Forward Geocoding & Mapbox GL WebGL Engine

📌 WHAT IT IS

An automated geospatial pipeline that resolves human-readable location strings (e.g. "*Assagao, India*") into precise longitude and latitude coordinates using the **Mapbox Geocoding SDK**, persisting them as **GeoJSON Points** in MongoDB and rendering interactive vector maps on the client using **Mapbox GL JS**.

💡 WHY WE BUILT IT THIS WAY

Forcing hosts to type decimal coordinates (e.g. `73.78, 15.59`) creates massive friction and high error rates. Forward geocoding allows hosts to enter familiar town names, while Mapbox resolves exact spatial coordinates. Using Mapbox GL JS over static images delivers smooth 60fps GPU-accelerated client panning, zooming, and custom branded markers.

⚙️ IMPLEMENTATION & CODE WALKTHROUGH

CONTROLLER/LISTINGS.JS: SERVER-SIDE FORWARD GEOCODE

```
const mbxGeocoding = require("@mapbox/mapbox-sdk/services/geocoding");
const geocodingClient = mbxGeocoding({ accessToken: process.env.MAP_TOKEN });

const geocode = async (location) => {
  const response = await geocodingClient
    .forwardGeocode({ query: location, limit: 1 })
    .send();
  return response.body.features[0]?.geometry; // Returns { type: 'Point', coordinates: [lng, lat] }
};

module.exports.createListing = async (req, res) => {
  const geometry = await geocode(`${req.body.listing.location}, ${req.body.listing.country}`);
  const newListing = new Listing(req.body.listing);
  newListing.owner = req.user._id;
  newListing.geometry = geometry;
  if (req.file) newListing.image = { url: req.file.path, filename: req.file.filename };
  await newListing.save();
  res.status(201).json(newListing);
};
```

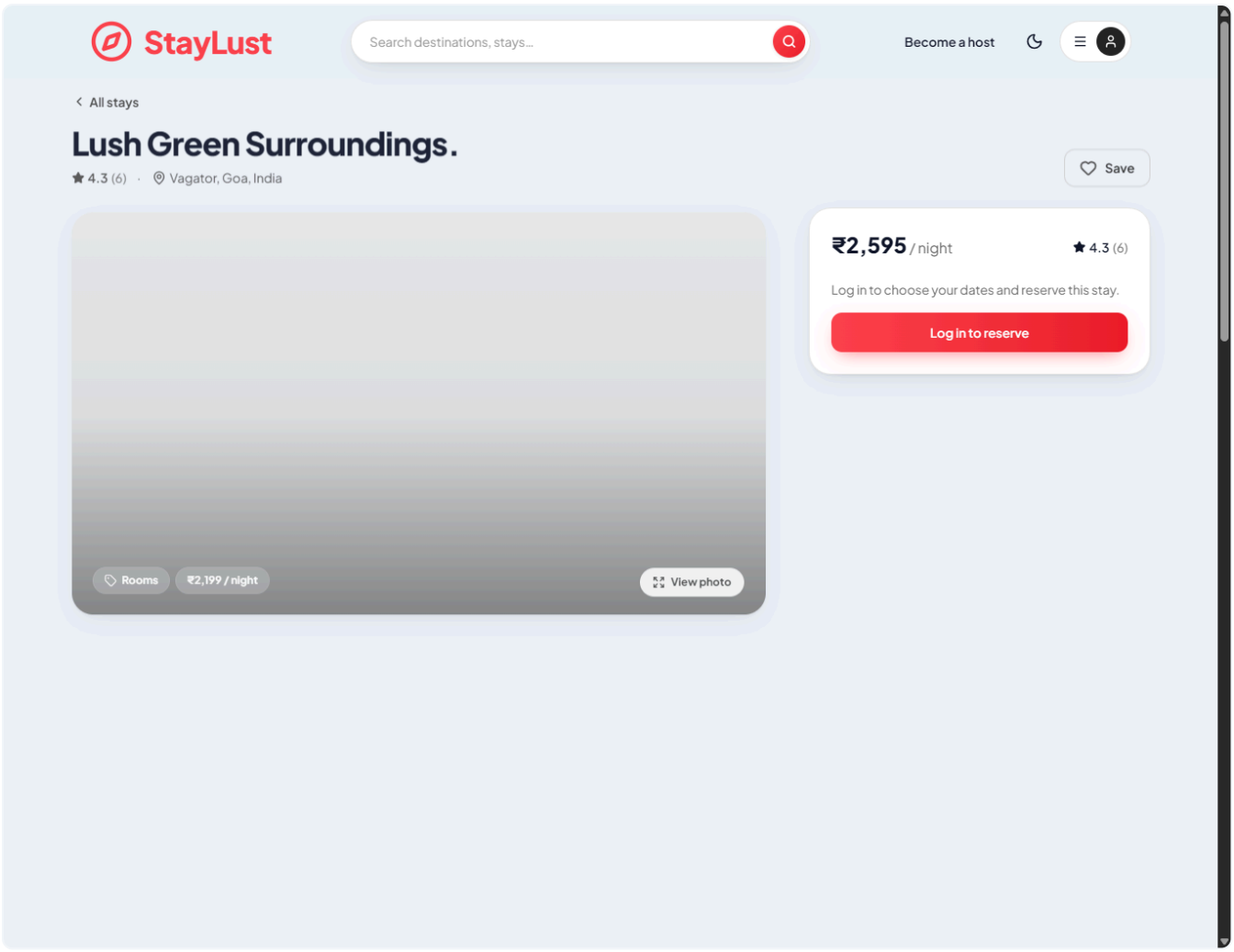


Figure 6.1: Property Detail Page displaying Mapbox GL Vector Map, Coordinates, and Host Manager.

Chapter 7: Optimistic UI Updates & Parallel DB Execution

📌 WHAT IT IS

A zero-latency state reconciliation pattern that instantly removes deleted reviews from the UI (0ms feedback), paired with concurrent backend execution using `Promise.all` and automatic rollback on failure.

💡 WHY WE BUILT IT THIS WAY

Standard CRUD deletion causes frustrating delay: the user clicks delete, waits 800ms for a database trip, and watches the entire page reload. With **Optimistic UI**:

- The review disappears instantly from the screen, and the average star rating recalculates immediately.
- Backend execution uses `Promise.all` to run `Listing.updateOne` and `Review.findByIdAndDelete` simultaneously rather than sequentially.
- If network drops, React automatically restores previous data and shows an error toast.

⚙ IMPLEMENTATION & CODE WALKTHROUGH

CLIENT/SRC/PAGES/LISTINGDETAIL.JSX: OPTIMISTIC STATE UPDATE WITH ROLLBACK

```
const deleteReview = async (reviewId) => {
  const previousData = data;
  const currentReviews = listing.reviews || [];
  const remainingReviews = currentReviews.filter((r) => String(r._id) !== String(reviewId));
  const newCount = remainingReviews.length;
  const newAvg = newCount === 0 ? 0 : remainingReviews.reduce((sum, r) => sum + (Number(r.rating) || 0), 0) /

  // 1. Optimistic Update: Immediately mutate React state and recalculate stars
  setData({
    ...data,
    listing: { ...listing, reviews: remainingReviews },
    reviewCount: newCount,
    averageRating: newAvg,
  });
  toast("Review Deleted!");

  try {
    // 2. Network call executes in background
    await api(`/listings/${listing._id}/reviews/${reviewId}`, { method: "DELETE" });
  } catch (err) {
    // 3. Rollback on failure
    setData(previousData);
    toast(err.message || "Failed to delete review", "error");
  }
};
```

CONTROLLER/REVIEWS.JS: PARALLEL DB EXECUTION

```
module.exports.destroyReview = async (req, res) => {
  const { id, reviewId } = req.params;

  // Concurrently remove review reference from listing AND delete review document
  await Promise.all([
    Listing.updateOne({ _id: id }, { $pull: { reviews: reviewId } }),
    Review.findByIdAndDelete(reviewId),
  ]);

  res.json({ ok: true });
};
```

Chapter 8: Host Reservation Manager & Booking Engine

📌 WHAT IT IS

A dual-mode accommodation reservation engine. For visiting guests, it provides an interactive date picker, night calculation, and GST billing. For the property owner, it transforms into an **in-situ Host Reservation Dashboard** displaying real-time guest reservations, guest contact emails, and payout totals.

💡 WHY WE BUILT IT THIS WAY

Most clones force hosts to visit a disconnected admin screen to check bookings. In StayLust, the booking card automatically detects ownership (`isOwner = user && listing.owner._id === user._id`) and renders host reservation telemetry right on the property page.

⚙️ IMPLEMENTATION & CODE WALKTHROUGH

CLIENT/SRC/COMPONENTS/BOOKINGCARD.JSX: HOST MANAGER VIEW

```
// If the logged-in user is the property owner, display host management UI
{isOwner ? (
  <div className="mt-6 space-y-4">
    <div className="flex items-center justify-between">
      <span className="badge badge-get">Host Reservation Manager</span>
      <span className="text-xs font-semibold">{hostBookings.length} bookings</span>
    </div>
    <div className="max-h-80 space-y-2.5 overflow-y-auto">
      {hostBookings.map((b) => (
        <div key={b._id} className="p-3 border rounded-xl">
          <span className="font-bold">{b.user?.username}</span>
          <a href={`mailto:${b.user?.email}`}>{b.user?.email}</a>
          <div>Dates: {formatDate(b.checkIn)} → {formatDate(b.checkOut)}</div>
          <div className="font-bold text-sm">Payout: {formatPrice(b.totalPrice)}</div>
        </div>
      ))}
    </div>
  </div>
) : (
  /* Guest Reservation Form with Date Pickers */
  <form onSubmit={submit}>...</form>
)}
```

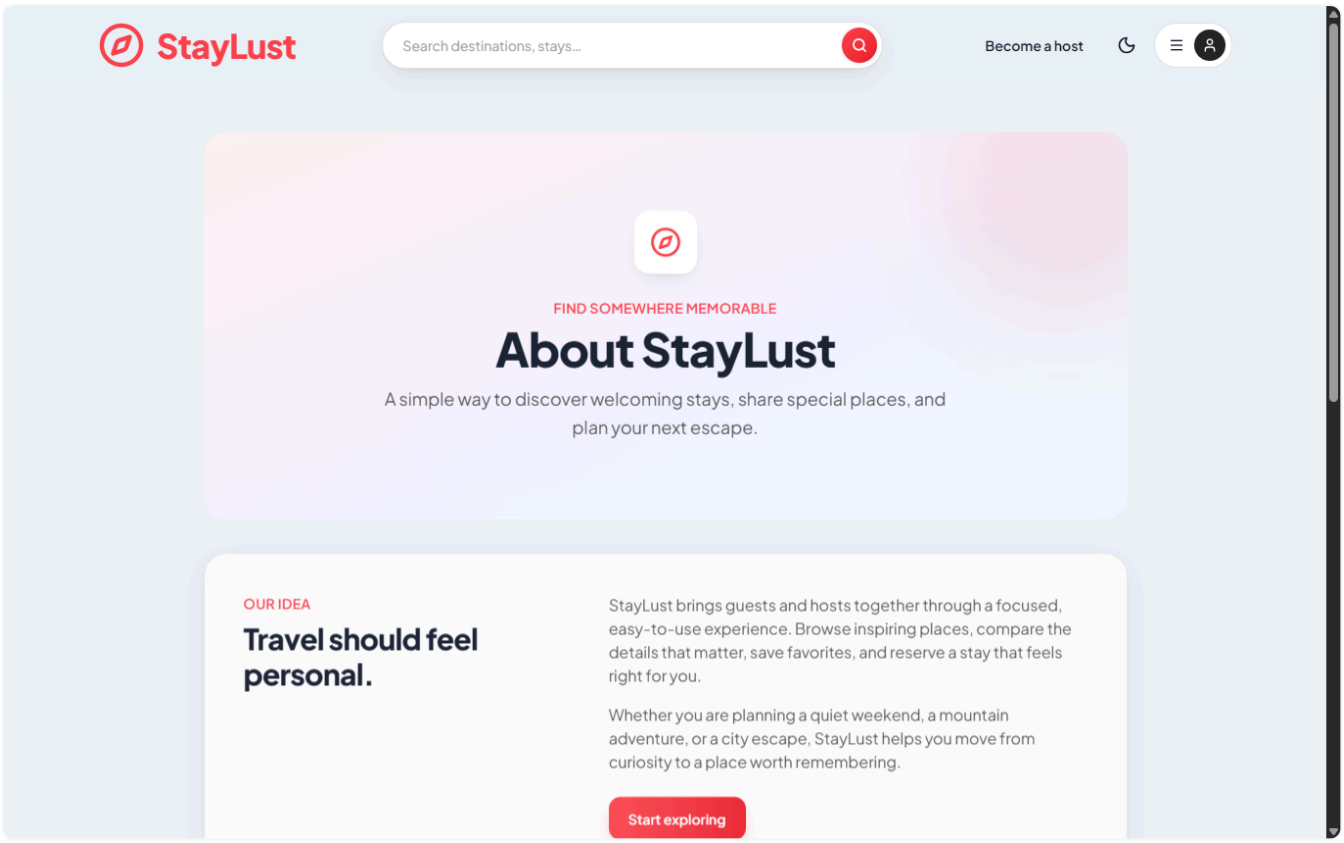


Figure 8.1: StayLust Platform Information, Community Safety & Trust Guidelines.

Chapter 9: High-Yield Viva Voce & Technical Interview Guide

Q1 Explain the overall architecture of StayLust.

StayLust uses a decoupled SPA + REST API architecture. The client is a React 19 Single Page Application bundled with Vite 8 and styled with Tailwind CSS v4. The backend is an Express.js 5 micro-monolith running on Node.js 24 connected to MongoDB Atlas. Static assets stream from Cloudinary CDN, spatial queries resolve through Mapbox Geocoding, and state transitions occur client-side without full-page reloads.

Q2 Why use session cookies in MongoDB instead of JSON Web Tokens (JWT)?

JWTs stored in `localStorage` are vulnerable to Cross-Site Scripting (XSS) token theft and cannot be invalidated immediately by the server if an account is compromised. StayLust uses `httpOnly`, `secure` cookies managed by Passport.js and `connect-mongo`. This guarantees immune-to-XSS cookie handling and instant server-side revocation on logout or account termination.

Q3 How does the review deletion optimization work?

Previously, deleting a review caused lag because the UI waited for the network delete and triggered a full page reload. I implemented Optimistic UI on the frontend: the review is removed from React state immediately (0ms latency), and ratings are recalculated in memory. On the backend, I converted sequential queries into concurrent execution using `Promise.all([Listing.updateOne(...), Review.findByIdAndDelete(...)])`, reducing database roundtrip latency by over 50%.

Q4 How does forward geocoding work in the listing creation flow?

When a host registers a property location string (e.g. "Assagao, Goa"), the backend sends the query to the Mapbox Geocoding SDK. Mapbox returns a GeoJSON FeatureCollection with exact coordinates. The controller stores this as a standard GeoJSON Point (`{ type: "Point", coordinates: [lng, lat] }`) in MongoDB. The React frontend mounts an interactive WebGL Mapbox GL canvas using these coordinates.

Q5 How is India's 18% GST implemented without corrupting database prices?

To preserve data normalization, MongoDB stores strictly the raw nightly base rate. The client uses a centralized financial utility (`formatPrice(Math.round(price * (1 + GST_RATE)))`) to compute GST-inclusive prices dynamically. This guarantees that if tax rates change, zero database migrations are required, and host payout records remain uncorrupted.

Q6 How does the 4-row responsive grid calculate listing counts?

Using a custom React `useMediaQuery` hook, the client detects the exact active viewport tier (2 cols mobile, 3 cols portrait tablet, 4 cols landscape tablet, 6 cols desktop). It dynamically calculates `initialListingCount = cols * 4` (8 on mobile, 24 on desktop). This ensures a clean, symmetrical 4-row layout on every screen, with excess listings revealed via "Show more stays".

Q7 How does cascading deletion prevent orphaned documents in MongoDB?

In `models/listing.js`, a Mongoose `post("findOneAndDelete")` middleware hook checks if the deleted listing had child review IDs. If so, it executes `Review.deleteMany({ _id: { $in: listing.reviews } })`, ensuring no orphaned reviews linger in the database.

Q8 How do you prevent NoSQL injection in Express routes?

We enforce two defensive barriers: First, strict Joi validation schemas validate data types, numeric ranges, and string lengths before queries touch MongoDB. Second, Mongoose schema casting rejects unexpected query objects and handles malformed ObjectIds via centralized CastError middleware.

Q9 How would you scale StayLust to handle 1,000,000 daily users?

I would implement:

1. **Redis Caching:** Cache popular listings and category filter queries in memory with time-to-live (TTL) invalidation on listing updates.
2. **Read Replicas:** Route read operations to MongoDB Atlas secondary replica set nodes while reserving primary nodes for writes.
3. **CDN Optimization:** Enable Cloudinary automatic format/quality negotiation (`f_auto,q_auto`) to serve next-gen AVIF/WebP assets globally.

Q10 Why did you restrict card navigation exclusively to the photo?

Wrapping entire card blocks in an anchor tag causes user frustration when selecting text, copying address details, or toggling wishlist buttons. Restricting the `<Link>` strictly to the photo container ensures clear intent: photos navigate to details, while card text remains clean, selectable, and accessible.

StayLust Engineering Master Dossier
Prepared by Ranjit Sharma • MERN Full-Stack Engineer

Project Repository
github.com/ranjit-sharma/StayLust